

DIGITAL LOGIC DESIGN

(CE_118)

CHAPTER 4:

PROCESSOR DESIGN

(part_1)

Outline

- **We introduced new concepts:**
 - ✓ Instruction sets
 - ✓ Instruction types
 - ✓ Addressing modes
 - ✓ Instruction-execution cycle
 - ✓ Processor design flow
- **Including**
 - ✓ instruction set design,
 - ✓ instruction set flowcharts,
 - ✓ component allocation,
 - ✓ ASM charts
 - ✓ processor architecture
- **We have demonstrated processor design**
 - ✓ 16-bit CISC design
 - ✓ 32-bit RISC design
 - data-forwarding
 - branch prediction

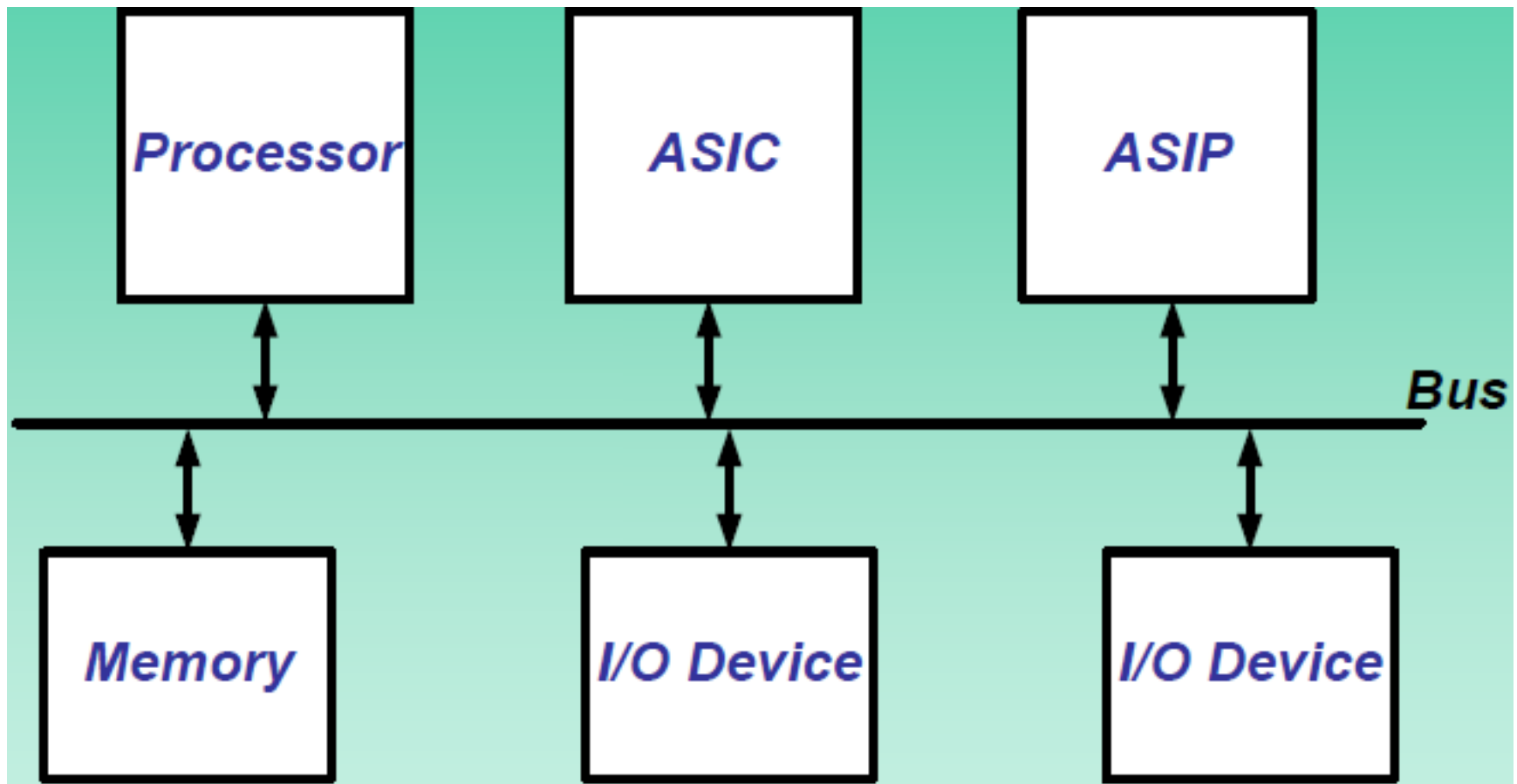
Outline

- **We introduced new concepts:**
 - ✓ **Instruction sets**
 - ✓ **Instruction types**
 - ✓ **Addressing modes**
 - ✓ **Instruction-execution cycle**
 - ✓ **Processor design flow**
- **Including**
 - ✓ **instruction set design,**
 - ✓ **instruction set flowcharts,**
 - ✓ **component allocation,**
 - ✓ **ASM charts**
 - ✓ **processor architecture**
- **We have demonstrated processor design**
 - ✓ **16-bit CISC design**
 - ✓ **32-bit RISC design**
 - **data-forwarding**
 - **branch prediction**

Basic definitions

- **Processor** controls overall system operations, supervising I/O devices (keyboards, monitors, disks, tapes), synchronize the data between system and components and perform most of computational tasks.
- **ASIC** (Application Specification Integrated Circuit) is a co-processor that executes one or more specific tasks much faster than the processor itself. For this reason, such ASICs are called accelerators, since the processor offloads computationally intensive tasks to them.
- **ASIP** (Application Specification Instruction Processor) is a processor that possesses *specific instructions* that will allow some applications to execute much faster than on an ordinary processor.

Basic computer architecture



Instruction Set

- **Instruction Set (IS)** is a set of variety of *instructions* and *instructions format*, which will be interpreted by the processor's control unit and executed in the processor's datapath.
- An instruction is a string of bits grouped into a number of different fields, such as
 - ✓ Operation code (op-code)
 - ✓ Instruction types
 - ✓ Addressing fields
- A typical instruction, such as, **a = b + c**, where a, b, and c are stored in memory at location A, B, and C can be expressed in assembly language format,
 - ✓ **Add A, B, C** (Mem[A] <- Mem[B]+ Mem[C])

(assembly instruction)

(mathematical notation)

Typical instructions: instruction-type field

- **Register instructions (arithmetic, logic, shift)**
 - ✓ **Add** RA, RB, RC ($RF[A] \leftarrow RF[B] + RF[C]$)
- **Move instructions (load and store)**
 - ✓ **Load** R2, A ($RF[2] \leftarrow MEM[A]$)
 - ✓ **Store** A, R2 ($MEM[A] \leftarrow RF[2]$)
- **Control instructions (branch instruction)**
 - ✓ **Beq** R2, R3, A $\left\{ \begin{array}{l} \text{if } R2=R3 \text{ then } PC \leftarrow MEM[A] \\ \text{if } R2 \neq R3 \text{ then } PC \leftarrow PC+1 \end{array} \right.$

Typical instructions: instruction-type field

➤ Mode field

Lind R2, A ($RF[2] \leftarrow Mem[Mem[A]]$)

➤ Constant field

Add R2, R3, 1 ($RF[2] \leftarrow RF[3] + 1$)

Number of address fields vs. performance

➤ Mathematical expression: $C = (a+b)(a-b)$

➤ Code with **three-address instructions**:

1. **Add** X, A, B ($\text{Mem}[X] \leftarrow \text{Mem}[A] + \text{Mem}[B]$)

2. **Sub** C, A, B ($\text{Mem}[C] \leftarrow \text{Mem}[A] - \text{Mem}[B]$)

3. **Mul** C, X, C ($\text{Mem}[C] \leftarrow \text{Mem}[X] \times \text{Mem}[C]$)

➤ Code size: 3 instructions

➤ Code performance:

- Assume memory sizes have 16 to 256 million words → each address would require 24 to 28 bits → 3-address in each instruction would require 80 to 90 bits (including the opcode field)

- Assume that processor has 32-bit data and memory with 32-bit words → each instruction address would occupy *three words* in memory

→ **9** memory accesses to fetch 3 instructions and **9** memory accesses to get the operand and store the result.

Number of address fields vs. performance

- Mathematical expression: $C = (a+b)(a-b)$
- Code with **two-address instructions** (with temporary data in memory):
 1. **Move** X, A ($\text{Mem}[X] \leftarrow \text{Mem}[A]$)
 2. **Add** X, B ($\text{Mem}[X] \leftarrow \text{Mem}[X] + \text{Mem}[B]$)
 3. **Move** C, A ($\text{Mem}[C] \leftarrow \text{Mem}[A]$)
 4. **Sub** C, B ($\text{Mem}[C] \leftarrow \text{Mem}[C] - \text{Mem}[B]$)
 5. **Mul** C, X ($\text{Mem}[C] \leftarrow \text{Mem}[C] \times \text{Mem}[X]$)
- Code size: 5 instructions
- Code performance: **10** memory accesses for instructions, **13** memory accesses for data

Number of address fields vs. performance

- Mathematical expression: $C = (a+b)(a-b)$
- Code with **one-address instructions** (with temporary data in the **accumulator** (ACC) register):
 1. **Load A** ($ACC \leftarrow Mem[A]$)
 2. **Add B** ($ACC \leftarrow ACC + Mem[B]$)
 3. **Store X** ($Mem[X] \leftarrow ACC$)
 4. **Load A** ($ACC \leftarrow Mem[A]$)
 5. **Sub B** ($ACC \leftarrow ACC - Mem[B]$)
 6. **Mul X** ($ACC \leftarrow ACC \times Mem[X]$)
 7. **Store C** ($Mem[C] \leftarrow ACC$)
- Code size: 7 instructions
- Code performance: **7** memory accesses for instructions, **7** memory accesses for data

Number of address fields vs. performance

- Mathematical expression: $C = (a+b)(a-b)$
- Code with **two-address instructions** (with temporary data in the **register file**):

1. Load R1, A	($RF[1] \leftarrow Mem[A]$)
2. Load R2, B	($RF[2] \leftarrow Mem[B]$)
3. Move R3, R2	($RF[3] \leftarrow RF[2]$)
4. Add R1, R2	($RF[1] \leftarrow RF[1] + RF[2]$)
5. Sub R3, R2	($RF[3] \leftarrow RF[3] - RF[2]$)
6. Mul R1, R3	($RF[1] \leftarrow RF[1] \times RF[3]$)
7. Store C, R1	($Mem[C] \leftarrow RF[1]$)
- Code size: 7 instructions
- Code performance: **7** memory accesses for instructions,
3 (Load x 2, Store) memory accesses for data

Number of address fields vs. performance

- Mathematical expression: $C = (a+b)(a-b)$
- Code with **three-address register instructions** and **two-address memory instructions** (with temporary data in the register file):
 1. **Load** R1, A ($RF[1] \leftarrow Mem[A]$)
 2. **Load** R2, B ($RF[2] \leftarrow Mem[B]$)
 3. **Add** R3, R1, R2 ($RF[3] \leftarrow RF[1] + RF[2]$)
 4. **Sub** R4, R1, R2 ($RF[4] \leftarrow RF[1] - RF[2]$)
 5. **Mul** R5, R4, R3 ($RF[5] \leftarrow RF[3] \times RF[4]$)
 6. **Store** C, R5 ($Mem[C] \leftarrow RF[5]$)
- Code size: 6 instructions
- Code performance: 6 memory accesses for instructions, 3 memory accesses for data

Addressing modes

Implied

(set, reset ACC/status registers)

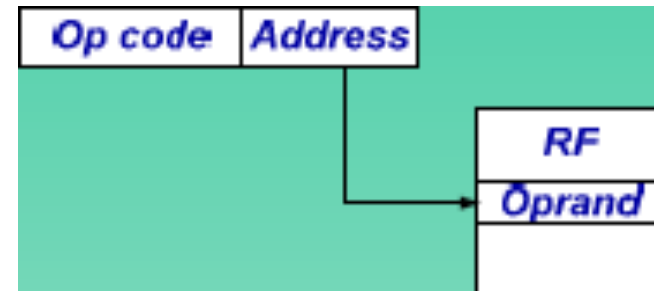
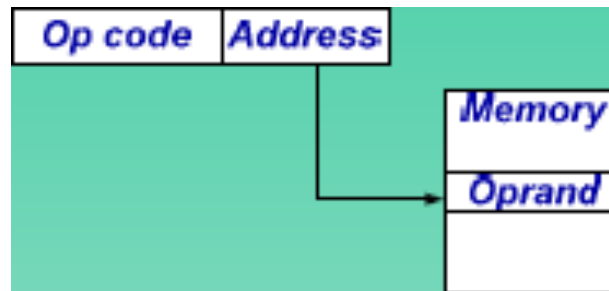
<i>Op code</i>

Immediate

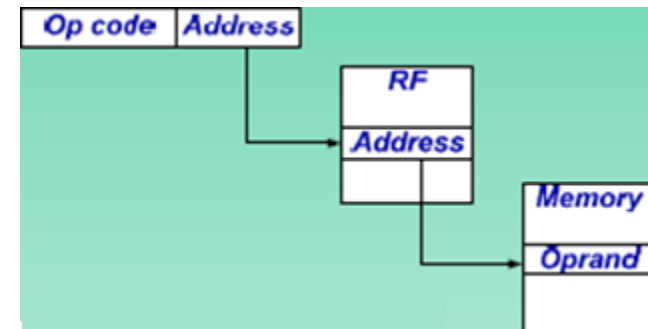
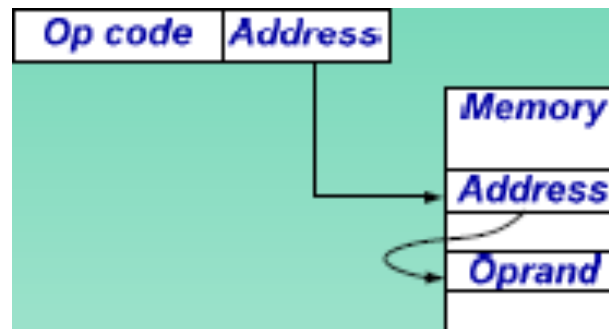
(incre/decre loop,
array indices, coefficient)

<i>Op code</i>	<i>Oprand</i>
----------------	---------------

Direct

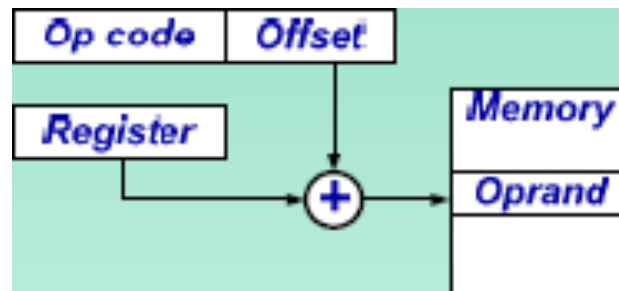


Indirect



Addressing modes

Relative



Branch instructions

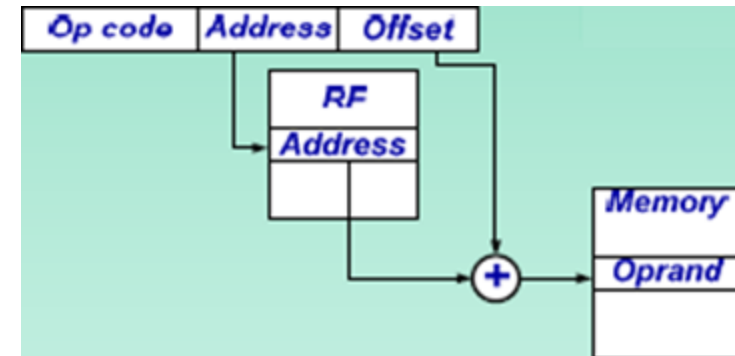
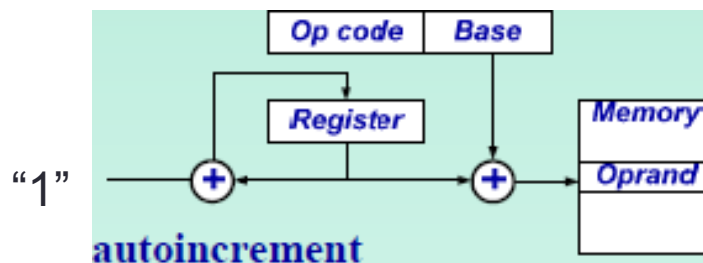
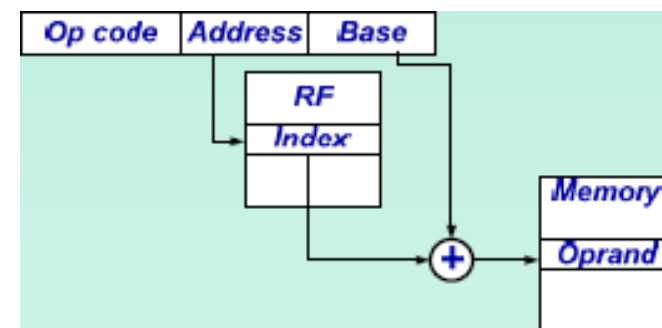


Table look-ups

Indexed



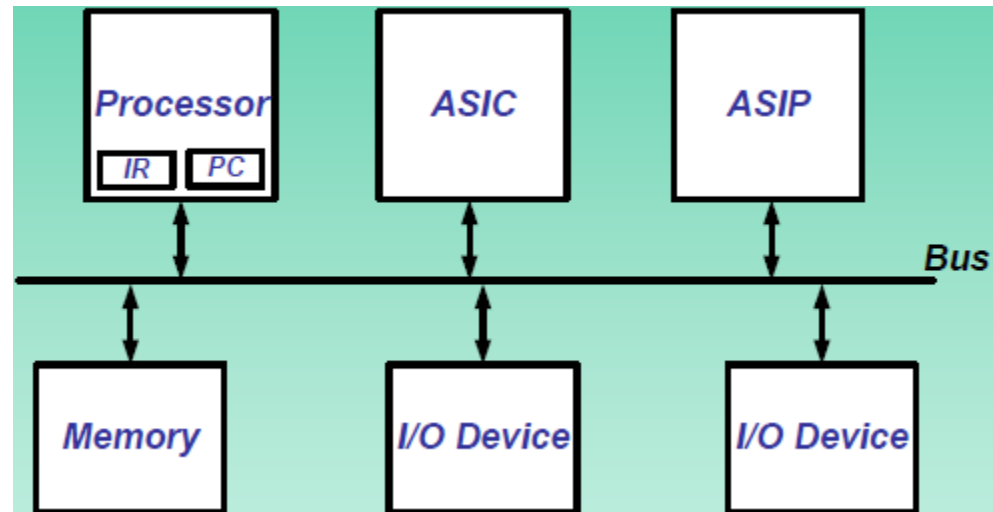
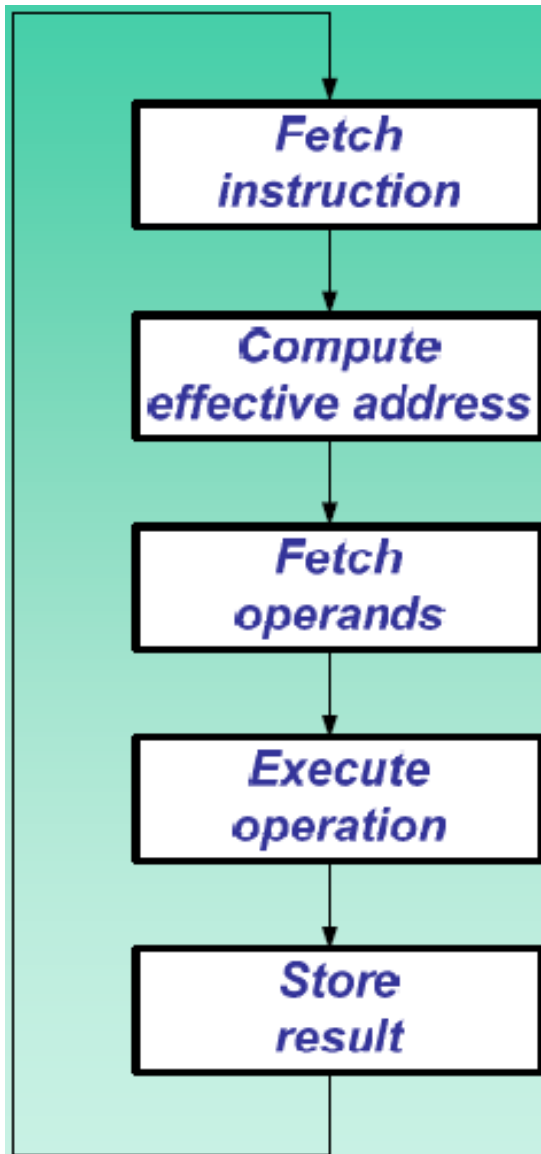
Access stack or queue



Access array

What is the difference between "Relative" and "Indexed" address?

Instruction-execution cycle



IR = Instruction Register

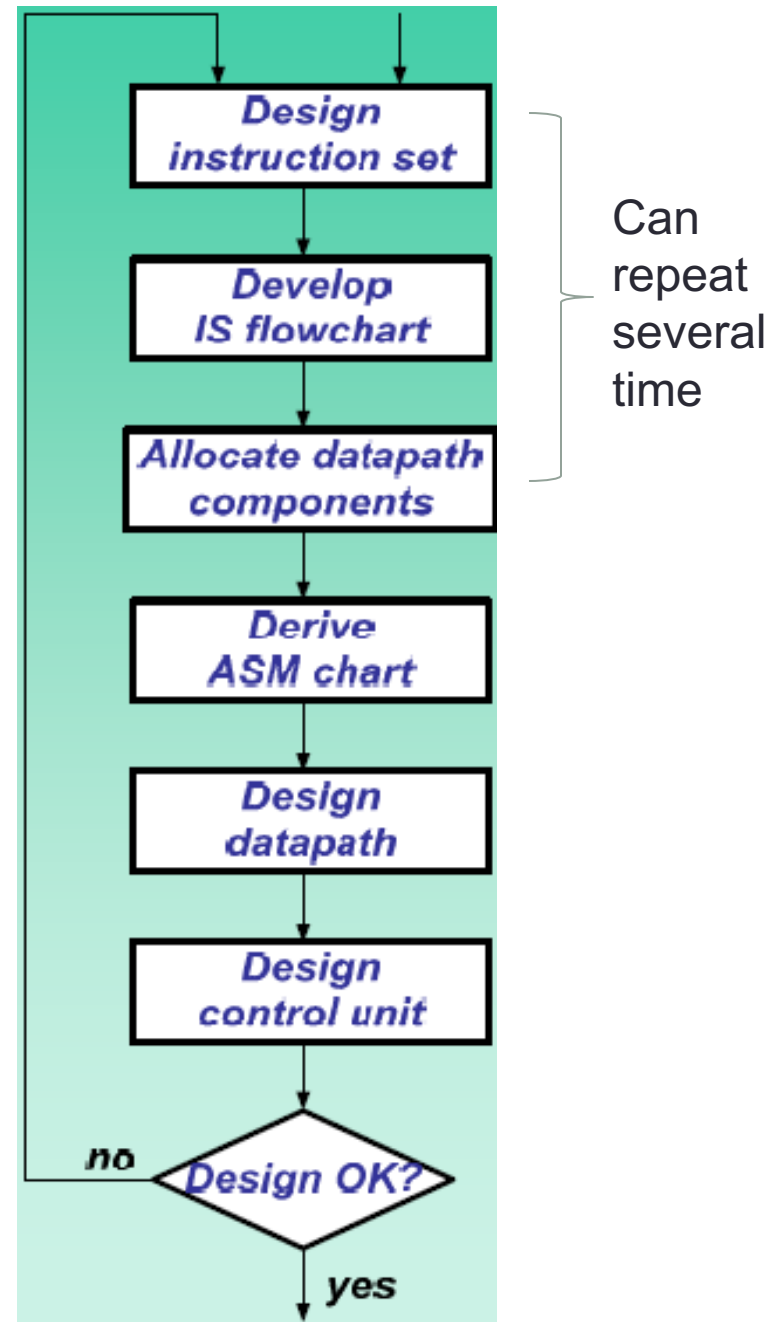
PC = Program Counter

Memory stores all instructions and data

Processor design flow

Trade-offs between **programming efficiency** and **program size** with **processor cost (size)** and **performance**

- ❖ **Instruction Set (IS)**: specify the operations taken by each instruction.
- ❖ **IS flowchart**: describe precisely all the operations that are performed in each instruction.
- ❖ **ASM chart**: divide each instruction into clock cycle.
- ❖ **Design datapath**: complete the connection.



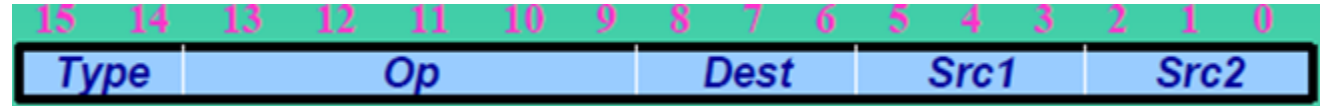
Instruction-set design

- Programming efficiency vs. program size
- Processor cost vs. processor performance
- Compromise **program size vs. processor size**
- **Complex Instruction-set (CISC)**
 - ✓ powerful instructions -> shorter programs
 - ✓ powerful instructions -> complex datapath, control unit
 - ✓ complex instructions -> several clock cycles
 - ✓ complex datapath, control unit -> longer clock period
 - ✓ complex instructions -> poor pipeline
- **Reduced Instruction-set (RISC)**
 - ✓ simple instructions -> longer programs
 - ✓ simple instructions -> simple datapath, control unit
 - ✓ simple instructions -> single clock cycle
 - ✓ simple datapath -> shorter clock period
 - ✓ simple instructions -> excellent pipeline

Complex instruction-set for a 16-bit processor

Each instruction  **1 word 16-bit** (no memory access) (2-bit Type field, 5-bit Op field,...)
2 words 16-bit (memory access in word 2) → access **64K** memory

a) **Register instructions**
 arithmetic, logic,
 move and shift



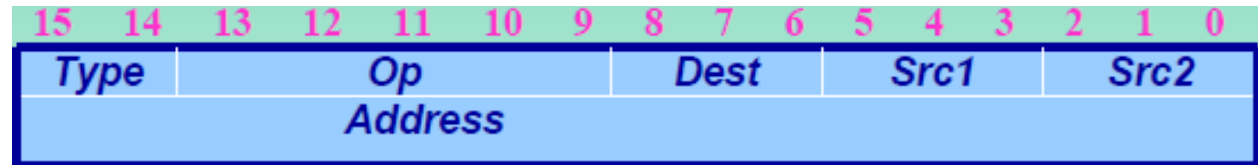
Name	Action
Op Dest, Src1, Src2	$RF(Dest) \leftarrow RF(Src1) \text{ Op } RF(Src2)$

b) **Memory instructions**
 load and store



Name	Name Action
L imm Dest	$RF[Dest] \leftarrow Address$
L dir Dest	$RF[Dest] \leftarrow Mem[Address]$
L rel Dest, Src2	$RF[Dest] \leftarrow Mem[RF[Src2]+Address]$
L in Dest	$RF[Dest] \leftarrow Mem[Mem[Address]]$
S dir Src1	$Mem[Address] \leftarrow RF[Src1]$
S rel Src1, Src2	$Mem[RF[Src2]+Address] \leftarrow RF[Src1]$
S in Src1	$Mem[Mem[Address]] \leftarrow RF[Src1]$

Complex instruction-set for a 16-bit processor



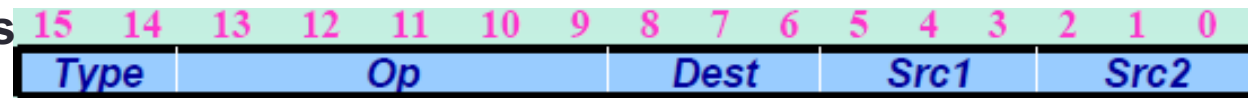
c) Control instructions

jump, branch, call and return

Name	Action
Jump Address	$PC \leftarrow \text{Address}$
Brel Address	$PC \leftarrow PC+1$ if $\text{Status}[\text{rel}] = 0$ $PC \leftarrow \text{Address}$ if $\text{Status}[\text{rel}] = 1$
Call Address, Src1	$\text{Mem}[\text{RF}[\text{Src1}]] \leftarrow PC+1$; $PC \leftarrow \text{Address}$; $\text{RF}[\text{Src1}] \leftarrow \text{RF}[\text{Src1}] + 1$
Return	$\text{RF}[\text{Src1}] \leftarrow \text{RF}[\text{Src1}] - 1$; $PC \leftarrow \text{Mem}[\text{RF}[\text{Src1}]]$;

d) Miscellaneous instructions

no-op, clear, status, set and reset

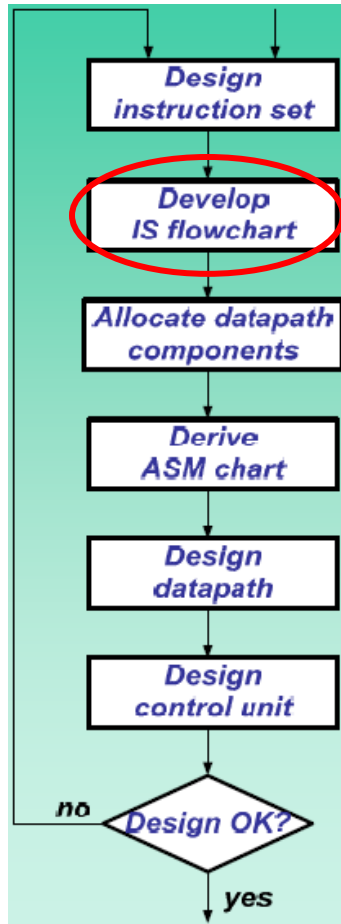


Name	Action	six relations
No-op	Do nothing	
Clear Dest	$\text{RF}[\text{Dest}] \leftarrow 0$	
Lstat Src1, Src2	$\text{Status} \leftarrow \text{R}[\text{Src1}] \text{ >=< } \text{RF}[\text{Src2}]$	↖
Sstat Dest	$\text{Status}[\text{Dest}] \leftarrow 1$	
Rstat Dest	$\text{Status}[\text{Dest}] \leftarrow 0$	

Outline

- We introduced new concepts:
 - ✓ Instruction sets
 - ✓ Instruction types
 - ✓ Addressing modes
 - ✓ Instruction-execution cycle
 - ✓ Processor design flow
- Including
 - ✓ instruction set design,
 - ✓ instruction set flowcharts,
 - ✓ component allocation,
 - ✓ ASM charts
 - ✓ processor architecture
- We have demonstrated processor design
 - ✓ 16-bit CISC design
 - ✓ 32-bit RISC design
 - data-forwarding
 - branch prediction

CISC Design: Instruction-set flowchart



Processor design flow

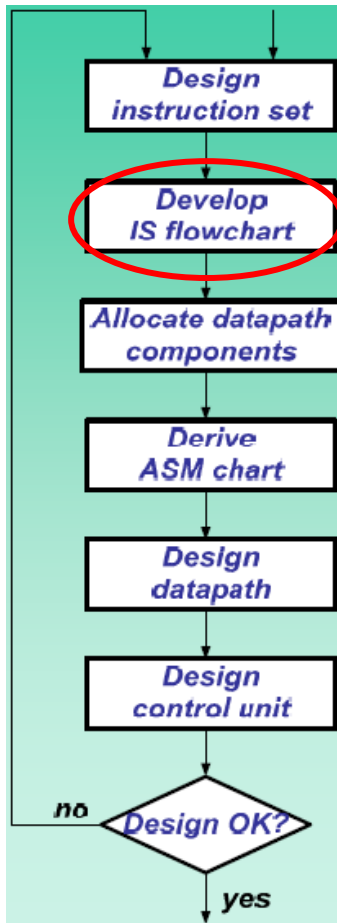
Instruction-set flowchart

- Does not presume any architectural details
- Does not presume any particular processor datapath
- Does not consider any timing constraints or clock cycle duration

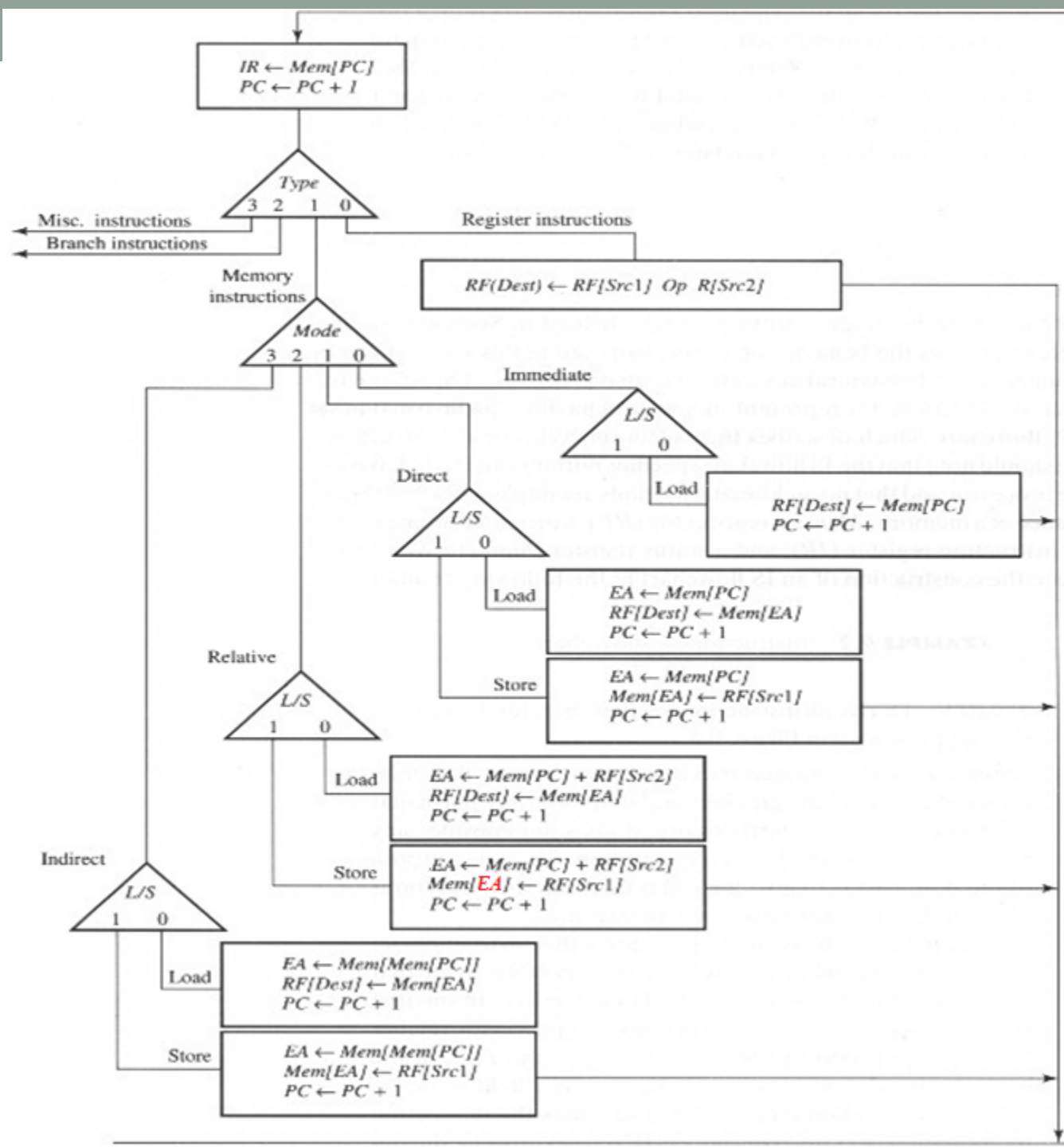
Purpose:

- Give the order in which the operations specified by each instruction will be executed.

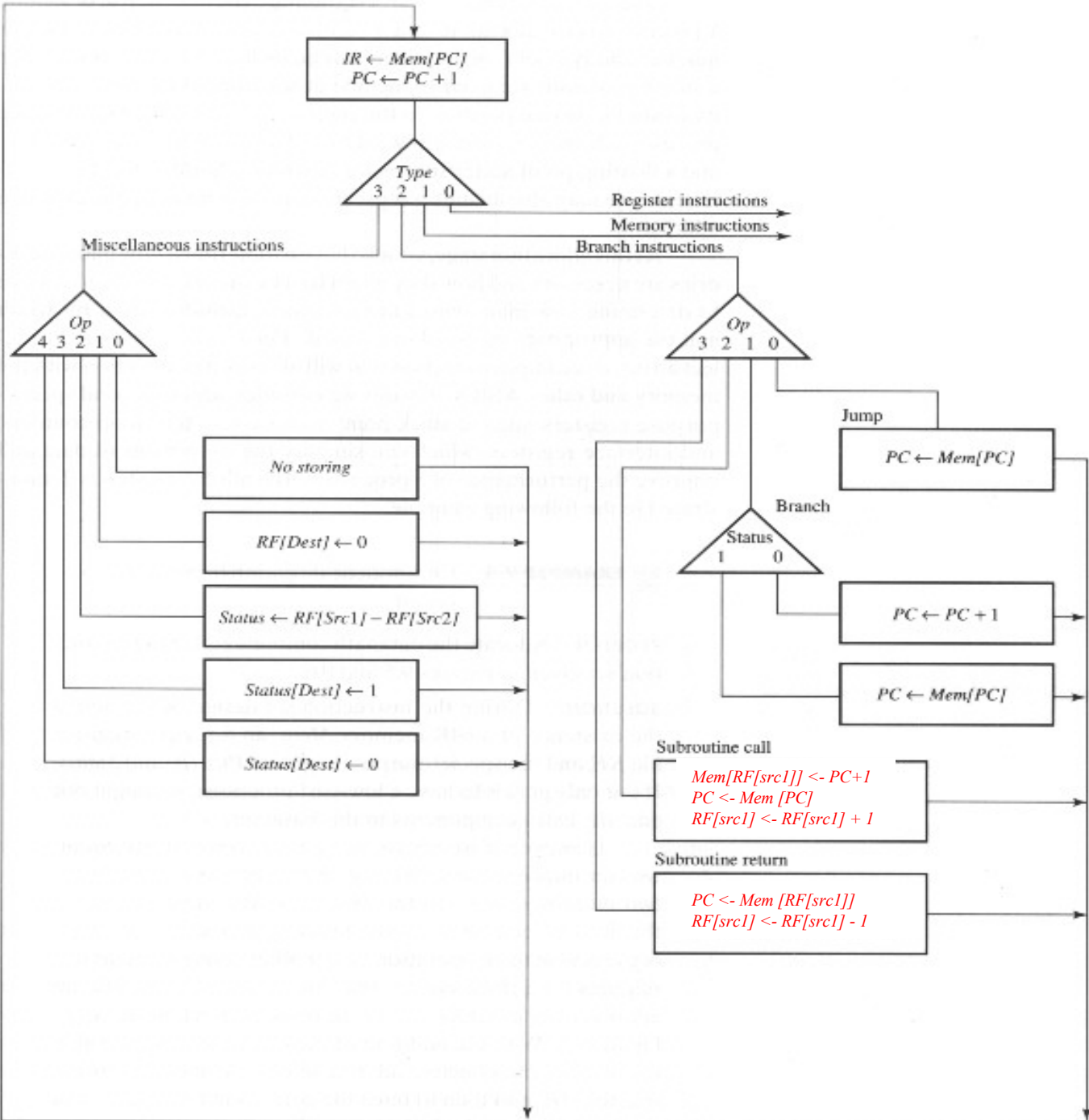
CISC Design: Instruction-set flowchart



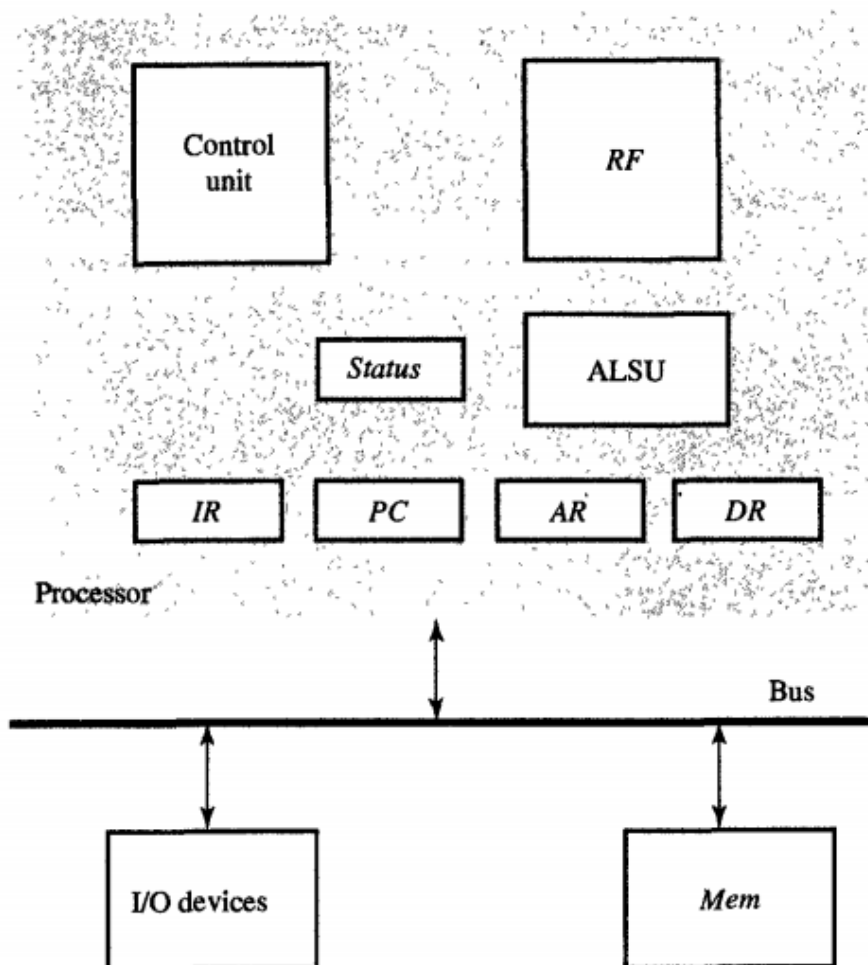
Design process



CISC Design:
Instruction-set
flowchart
(cont.)



Component allocation for the 16-bit processor



Components: 64K x 16 Memory

8 x 16 Register file

ALSU

Instruction register (IR)

Program counter (PC)

Address register (AR)

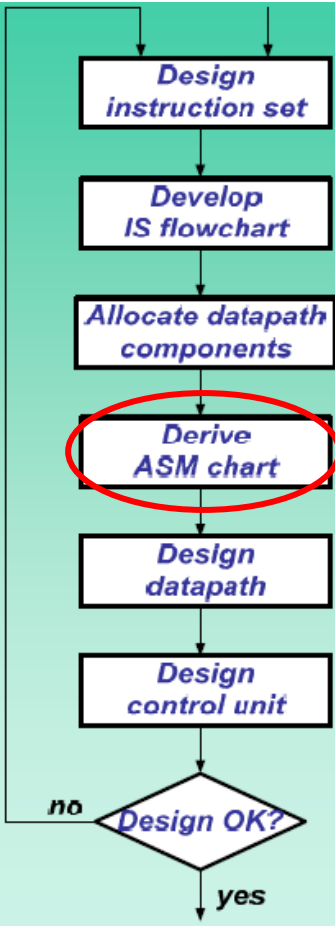
Data register (DR)

Status register (Status)

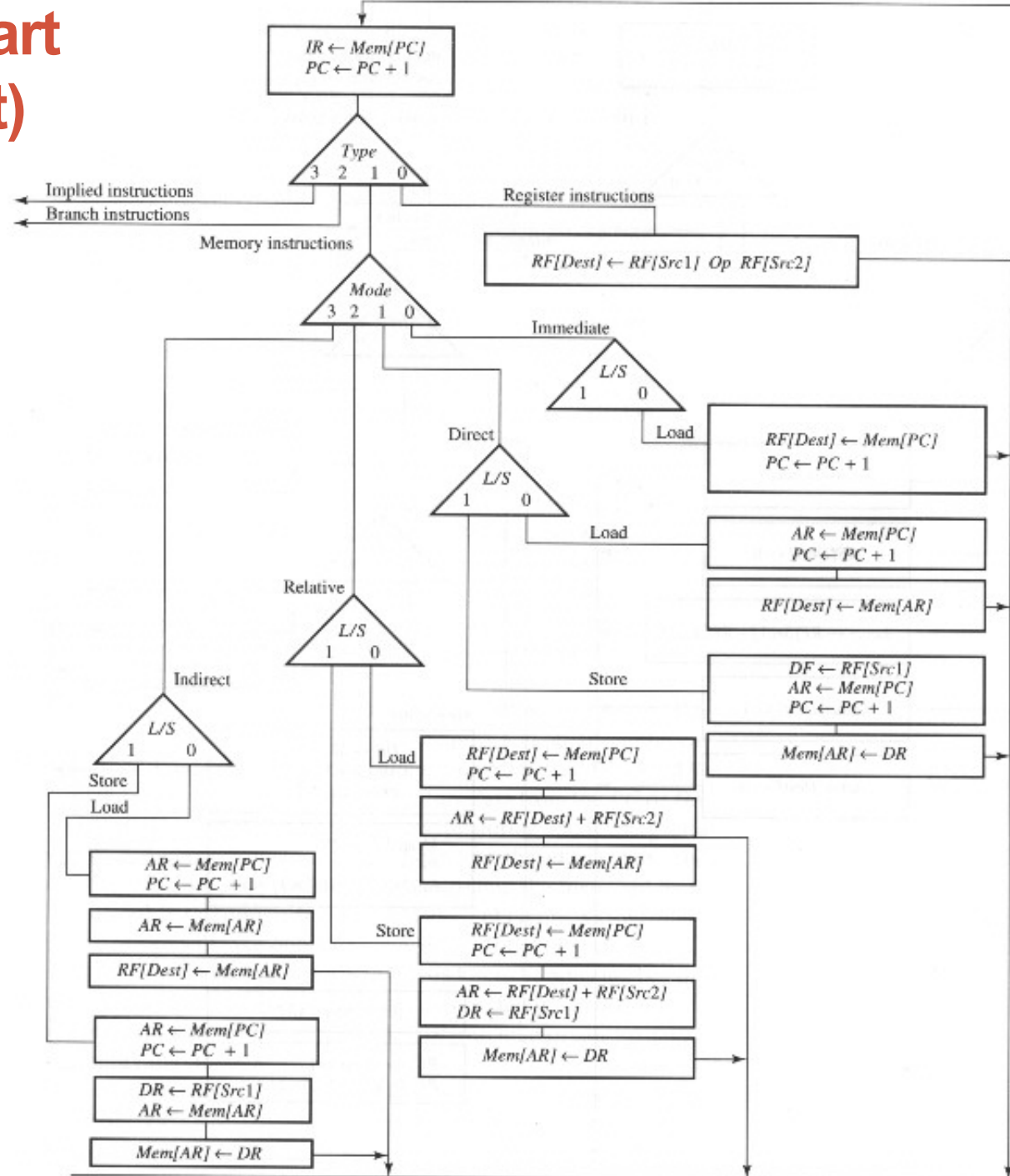
Control unit

AR, DR: needed to shorten clock period and improve the performance of the processor

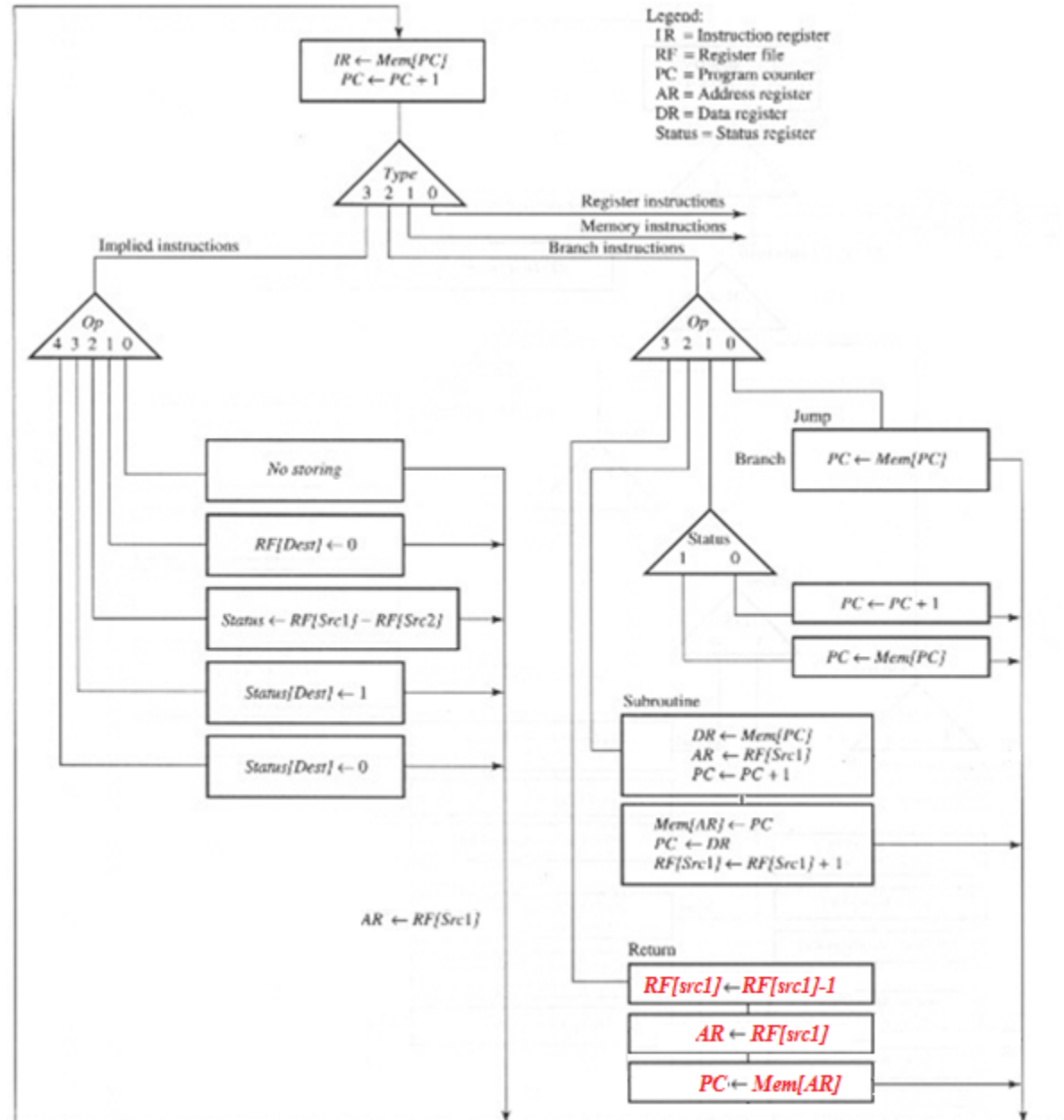
Processor ASM chart (scheduled IS chart)



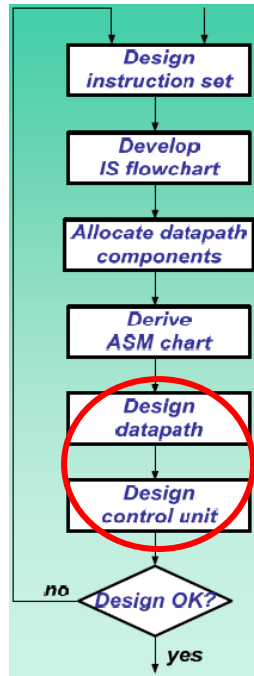
Design process



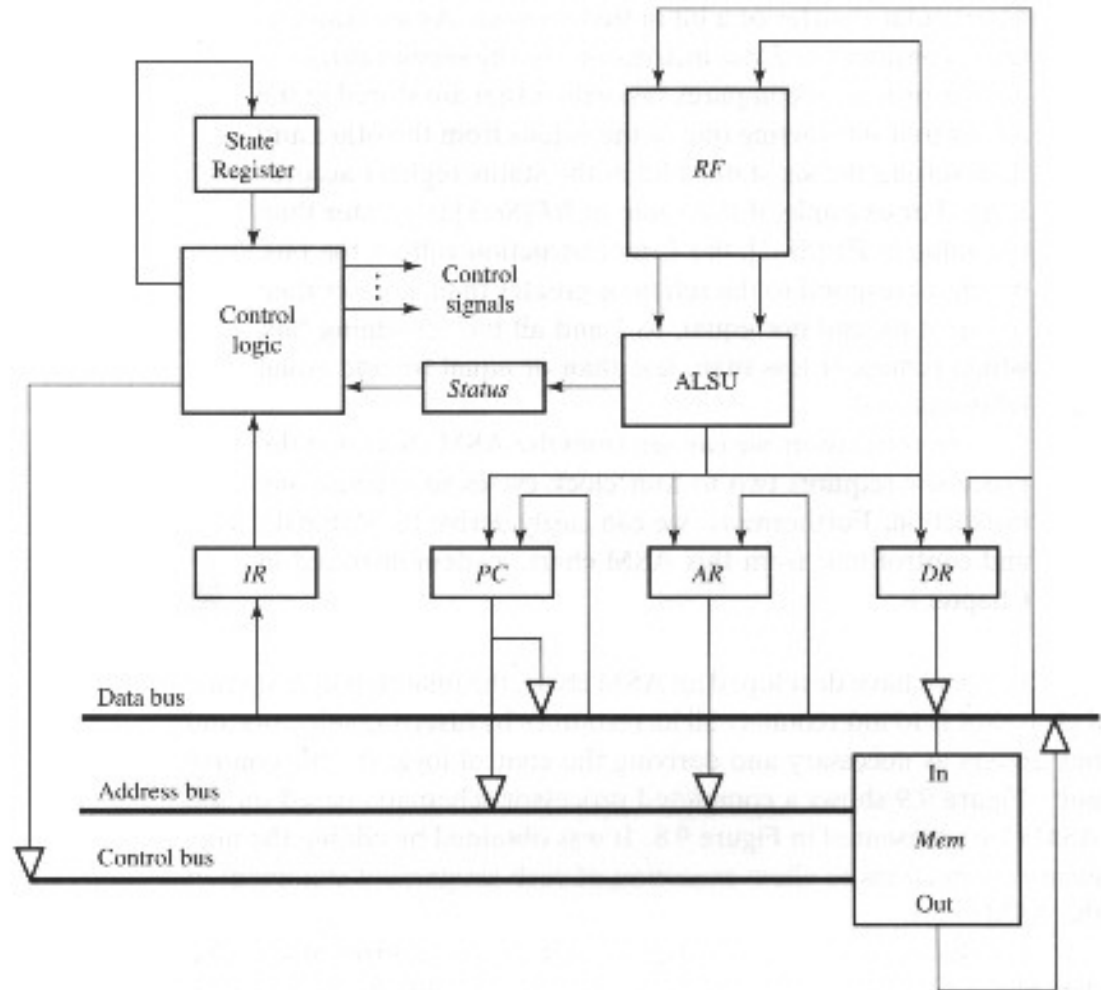
Processor ASM chart (scheduled IS chart)



Processor schematic



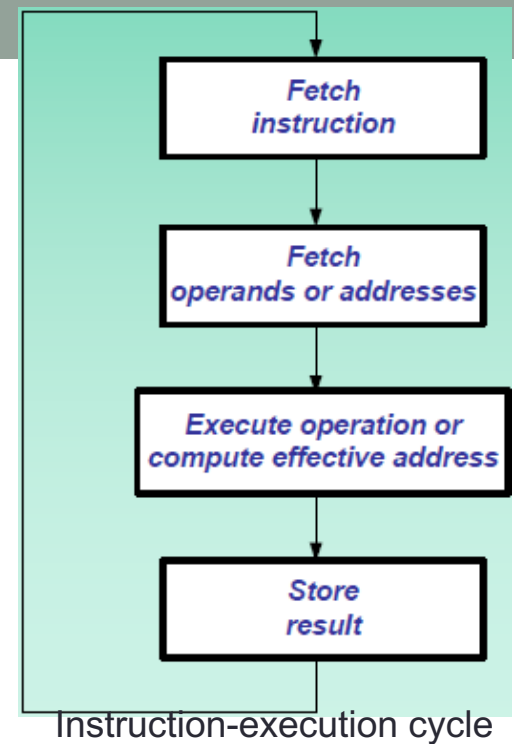
Design process



- Schematic is obtained according to the ASM chart, that is, by adding a connection whenever data or an instruction is moved from one component to another.
- Components with several connections at the same input port require selectors for that particular input port.

Reduced instruction-set cycle (RISC design)

- Register and miscellaneous instructions do not need address fetch and effective address computation
- Memory and control instructions do not need operand fetch and operation execution
- Share operand fetch and address fetch
- Share operand execution and effective address computation
- Thus, instruction cycle reduced to 4 steps



Clock cycle	0	1	2	3	4	5
Fetch instruction	I_1	I_2	I_3	I_4	I_5	I_6
Fetch operands or addresses		I_1	I_2	I_3	I_4	I_5
Execute operation or compute effective address			I_1	I_2	I_3	I_4
Store result				I_1	I_2	I_3

Pipelined execution

Reduced instruction-set for a 32-bit processor (RISC type)

a) Register Instructions

arithmetic, logic, move
and shift



<u>Name</u>	<u>Action</u>
Op Dest, Src1, Src2	$RF(Dest) \leftarrow RF[Src1] \text{ Op } RF[Src2]$
Op Dest, Src1, Src2	$RF(Dest) \leftarrow RF[Src1] \text{ Op } Constant$
Move Dest, Src1	$RF(Dest) \leftarrow RF[Src1]$
Shift Dest, Src1, Constant	$RF(Dest) \leftarrow RF[Src1] \text{ shift } Constant$



b) Memory instructions

load and store

2 addressing mode:

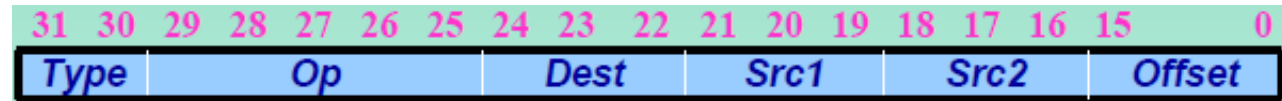
- Immediate
- Relative

<u>Name</u>	<u>Action</u>
L immU Dest	$RF[Dest(31...16)] \leftarrow Offset$
L immL Dest	$RF[Dest(15...0)] \leftarrow Offset$
L rel Dest, Src2, Offset	$RF[Dest] \leftarrow Mem[RF[Src2] + Offset]$
S rel Src1, Src2, Offset	$Mem[RF[Src2] + Offset] \leftarrow RF[Src1]$

Reduced instruction-set for a 32-bit processor (RISC type)

c) Control instructions

jump and branch



Beq (=)

Bgre (>)

Bgoeq ($\geq$)

Bless (<)

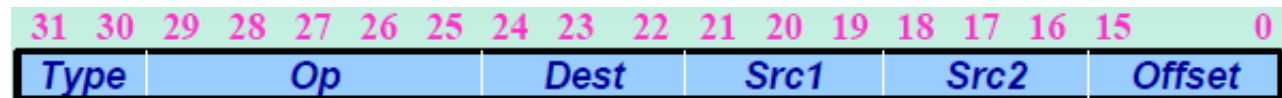
Bloeq ($\leq$)

Bneq ($\neq$)

<u>Name</u>	<u>Action</u>
Jump Offset	$PC \leftarrow PC + \text{offset}$
Jump Src2, Offset	$PC \leftarrow RF[\text{Src2}] + \text{offset}$
Brel Src1, Src2, Offset	$PC \leftarrow PC+1$ if $RF[\text{Src1}]$ not rel $RF[\text{Src2}]$ $PC \leftarrow PC+\text{Offset}$ if $RF[\text{Src1}]$ rel $RF[\text{Src2}]$

d) Miscellaneous instructions

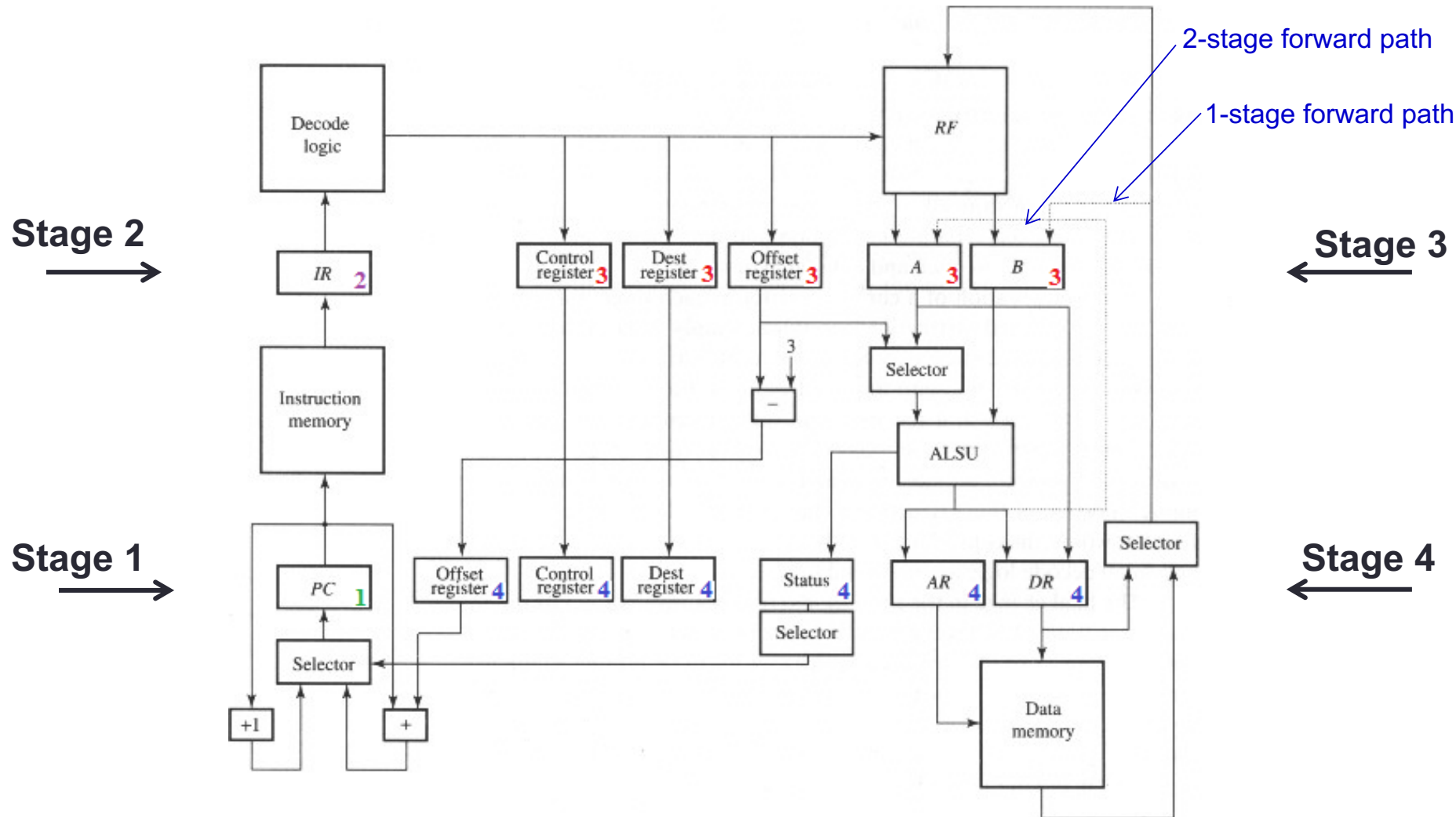
no-op, clear,
set and reset



<u>Name</u>	<u>Action</u>
No-op	Do nothing
Clear Dest	$RF[\text{Dest}] \leftarrow 0$
Sstat Dest	$\text{status}[\text{Dest}] \leftarrow 1$
Rstat Dest	$\text{status}[\text{Dest}] \leftarrow 0$

RISC block diagram

- Four-stage pipeline
- Separate instruction and data memories
- Add control register in each pipeline stage
- Pipeline stalling (flushing) for control instructions



RISC operation for a 3-line program

x = a + b

y = b - c

z = c + d

Source program

Concurrently:

$n+3$ clock cycles to execute n instructions

Sequentially:

$4n$ clock cycles for n instructions (no pipeline)

Address	Instruction
100	Load a, base, off
101	Load b, base, off
102	Load c, base, off
103	Load d, base, off
104	Add x, a, b
105	Sub y, b, c
106	Add z, c, d
107	Store x, base, off
108	Store y, base, off
109	Store z, base, off
110	—

Assembly program

Clock cycle	0	1	2	3	4	5	6	7	8	9	10	11	12	13
Read PC	100	101	102	103	104	105	106	107	108	109	110			
Write IR	Load	Load	Load	Load	Add	Sub	Add	Store	Store	Store				
Write A						a	b	c	x	y	z			
Write B		base	base	base	base	b	c	d	base	base	base			
Write AR			base +off	base +off	base +off	base +off				base +off	base +off	base +off		
Write DR							a+b	b-c	c+d	x	y	z		
Write RF				a	b	c	d	x	y	z				
Write Mem											x	y	z	
Write PC	101	102	103	104	105	106	107	108	109	110	111	112	113	114

Timing diagram

RISC operation for a 2-line program with data dependencies

sum = a + b

total = sum + c

Source program

Data dependence:

5 (or 45%) of **11** instructions are **No-op** instructions → decrease substantially performance of pipelined processor

Address	Instruction
100	Load a, base, offa
101	Load b, base, offb
102	Load c, base, offc
103	No-op
104	Add sum, a, b
105	No-op
106	No-op
107	Add total, c, sum
108	No-op
109	No-op
110	Store total, base, offt

Assembly program

Clock cycle	0	1	2	3	4	5	6	7	8	9	10	11	12	13
Read PC	100	101	102	103	104	105	106	107	108	109	110			
Write IR	Load	Load	Load	No-op	Add	No-op	No-op	Add	No-op	No-op	Store			
Write A						a			Sum			total		
Write B		base	base	base		b			c			base		
Write AR			base +offa	base +offb	base +offc								base +offt	
Write DR							a+b			sum+c			total	
Write RF				a	b	c		sum			total			
Write Mem														total
Write PC	101	102	103	104	105	106	107	108	109	110	111	112	113	114

Timing diagram

RISC operation for a 2-line program with data-forwarding

sum = a + b
total = sum + c

Source program

Address	Instruction
100	Load a, base, off
101	Load b, base, off
102	Load c, base, off
103	Add sum, a, b
104	Add total, c, sum
105	Store total, base, off

Assembly program

Clock cycle	0	1	2	3	4	5	6	7	8
Read PC	100	101	102	103	104	105			
Write IR	Load	Load	Load	Add	Add	Store			
Write A					a	$F2[a+F1[B]]$	$F2[(a+F1[B]) + F1[C]]$		
Write B		base	base	base	$F1[B]$	$F1[C]$	base		
Write AR			base + off	base + off	base + off			base + off	
Write DR						$a + F1[B]$	$F2[a+F1[B]] + F1[C]$	$F2[(a+F1[B]) + F1[C]]$	
Write status									
Write RF				a	b	c	sum	total	
Write Mem									total
Write PC	101	102	103	104	105	106	107	108	109

Timing diagram

RISC operation for a 2-line program with data-forwarding

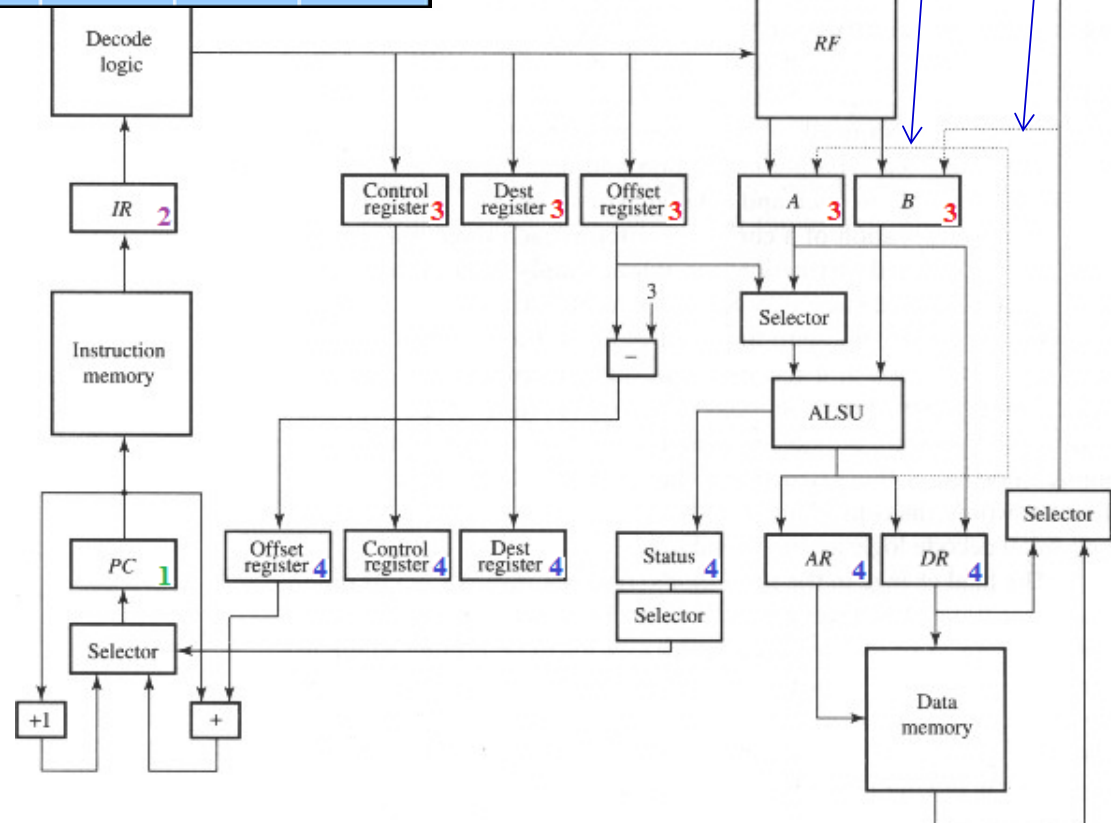
Clock cycle	0	1	2	3	4	5	6	7	8
Read PC	100	101	102	103	104	105			
Write IR	Load	Load	Load	Add	Add	Store			
Write A					a	$F2[a+F1[B]]$	$F2[(a+F1[B]) + F1[C]]$		
Write B		base	base	base	$F1[B]$	$F1[C]$	base		
Write AR			base + off	base + off	base + off			base + off	
Write DR						$a + F1[B]$	$F2[a+F1[B]] + F1[C]$	$F2[(a+F1[B]) + F1[C]]$	
Write status									
Write RF				a	b	c	sum	total	
Write Mem									total
Write PC	101	102	103	104	105	106	107	108	109

2-stage
forward path

1-stage
forward path

sum = a + b
total = sum + c

Address	Instruction
100	Load a, base, off
101	Load b, base, off
102	Load c, base, off
103	Add sum, a, b
104	Add total, c, sum
105	Store total, base, off



RISC operation without branch prediction

```

If a ≥ b then
  begin
    max = a
    min = b
  end
else
  begin
    max = b
    min = a
  end
endif

```

Source program

Address	Instruction
100	Bgoeq a, b, +10
101	No-op
102	No-op
103	No-op
104	Move max, b
105	Move min, a
106	Jump + 6
107	No-op
108	No-op
109	No-op
110	Move max, a
111	Move min, b
112	—

Assembly program

Clock cycle	0	1	2	3	4	5	6	7	8	9	10
Read PC	100	101	102	103	104	105	106	107	108	109	112
Write IR	Bgoeq	No-op	No-op	No-op	Move	Move	Jump	No-op	No-op	No-op	
Write A		a				b	a				
Write B		b									
Write AR											
Write DR							b	a			
Write status			a < b								
Write RF								max	min		
Write Mem											
Write PC	101	102	103	104	105	106	107	108	109	112	

Timing diagram when branch is not taken

Clock cycle	0	1	2	3	4	5	6	7	8	9	10
Read PC	100	101	102	103	110	111	112				
Write IR	Bgoeq	No-op	No-op	No-op	Move	Move					
Write A		a				a	b				
Write B		b									
Write AR											
Write DR							a	b			
Write status			a ≥ b								
Write RF								max	min		
Write Mem											
Write PC	101	102	103	110	111	112					

Timing diagram when branch is taken

RISC operation with branch prediction

```

If a ≥ b then
  begin
    max = a
    min = b
  end
else
  begin
    max = b
    min = a
  end
endif

```

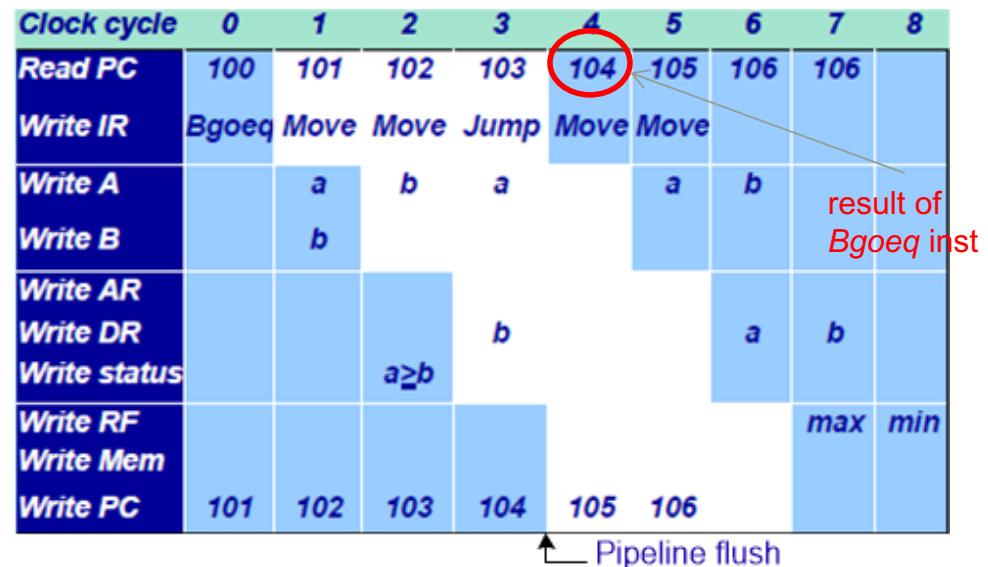
Source program

Address	Instruction
100	Bgoeq a, b, +4
101	Move max, b
102	Move min, a
103	Jump + 3
104	Move max, a
105	Move min, b
106	—

Assembly program



Timing diagram when branch is not taken



Timing diagram when branch is taken

Chapter Summary

- **We introduced new concepts:**
 - ✓ **Instruction sets**
 - ✓ **Instruction types**
 - ✓ **Addressing modes**
 - ✓ **Instruction-execution cycle**
 - ✓ **Processor design flow**
- **Including**
 - ✓ **instruction set design,**
 - ✓ **instruction set flowcharts,**
 - ✓ **component allocation,**
 - ✓ **ASM charts**
 - ✓ **processor architecture**
- **We have demonstrated processor design:**
 - ✓ **16-bit CISC design**
 - ✓ **32-bit RISC design**
 - **data-forwarding**
 - **branch prediction**